Google Cloud H2S

Gen AI Academy

APAC Edition

Build in APAC. Build for the world.

Participant Details

Participant name: Dhyan Jain

Build and deploy a single AI agent using ADK and Gemini that is hosted on Cloud Run and performs one clearly defined task.
The agent must be callable via an HTTP endpoint and return a valid response for a given input.
This is a mini project focused on agent structure and deployment, not complex logic.

Brief about the idea

Planet Guide Agent is an AI-powered conversational assistant built on Google's Agent Development Kit (ADK).

It enables users to explore and learn about planets, moons, and space missions in our solar system through a natural, multi-turn dialogue experience.

As documented in the A2A and ADK Working , The project uses a multi-agent workflow: a greeter agent welcomes the user and captures their query, a planetary researcher agent fetches scientific data using Wikipedia, and a response formatter agent presents findings as a professional "Mission Control" briefing.

Deployed on Google Cloud Run with a web UI, the agent is accessible to students, educators, and space enthusiasts — no setup required.

Meeting the Build Criteria

- How did you navigate the requirements of your chosen track to build a custom solution - using ADK, MCP, or AlloyDB AI that effectively addresses the 'What You Must Build' criteria?

Multi-Agent Architecture (ADK): Built using ADK's Agent and SequentialAgent classes to orchestrate a 3-agent pipeline — greeter, researcher, and formatter.

Tool Integration: Integrated LangChain's Wikipedia tool via ADK's LangchainTool wrapper, enabling real-time external knowledge retrieval.

State Management: Used ToolContext and output_key to share data across agents, enabling context-aware sequential processing.

Cloud Deployment: Deployed to Google Cloud Run using ADK's adk deploy cloud_run command with a web UI, service account, and proper IAM configuration.

Opportunities

- How different is it from any of the other existing ideas?
- How will it be able to solve the problem?
- USP of the proposed solution

Differentiation: Unlike static space encyclopaedias or generic chatbots, this agent dynamically fetches real-time Wikipedia data and presents it in a curated mission-briefing format — making learning immersive and contextual.

Problem Solved: instead of manually searching through the information , the project can itself access wikipedia to get the required data and articluate it based on user prompts

USP: A specialised multi-agent pipeline on ADK that separates research from presentation, ensuring both accuracy and engaging delivery — deployed live on Cloud Run with a public web UI.

List of features offered by the solution

Conversational Planetary Explorer: Ask questions about any planet or moon in natural language and receive detailed, educational answers.

Wikipedia-Powered Research: Live knowledge retrieval via LangChain's Wikipedia tool for up-to-date scientific facts on planets, moons, and orbital data.

Multi-Agent Sequential Pipeline: Separate specialist agents for research and response formatting, ensuring accuracy and quality of output.

Mission Control Tone: Responses presented in an inspiring, professional style making the experience engaging and memorable.

Cloud Run Deployment with Web UI: Fully deployed on Google Cloud Run with a publicly accessible web interface — no setup required.

Shared State Management: User queries persisted in agent state and passed seamlessly across the pipeline for coherent, context-aware responses.

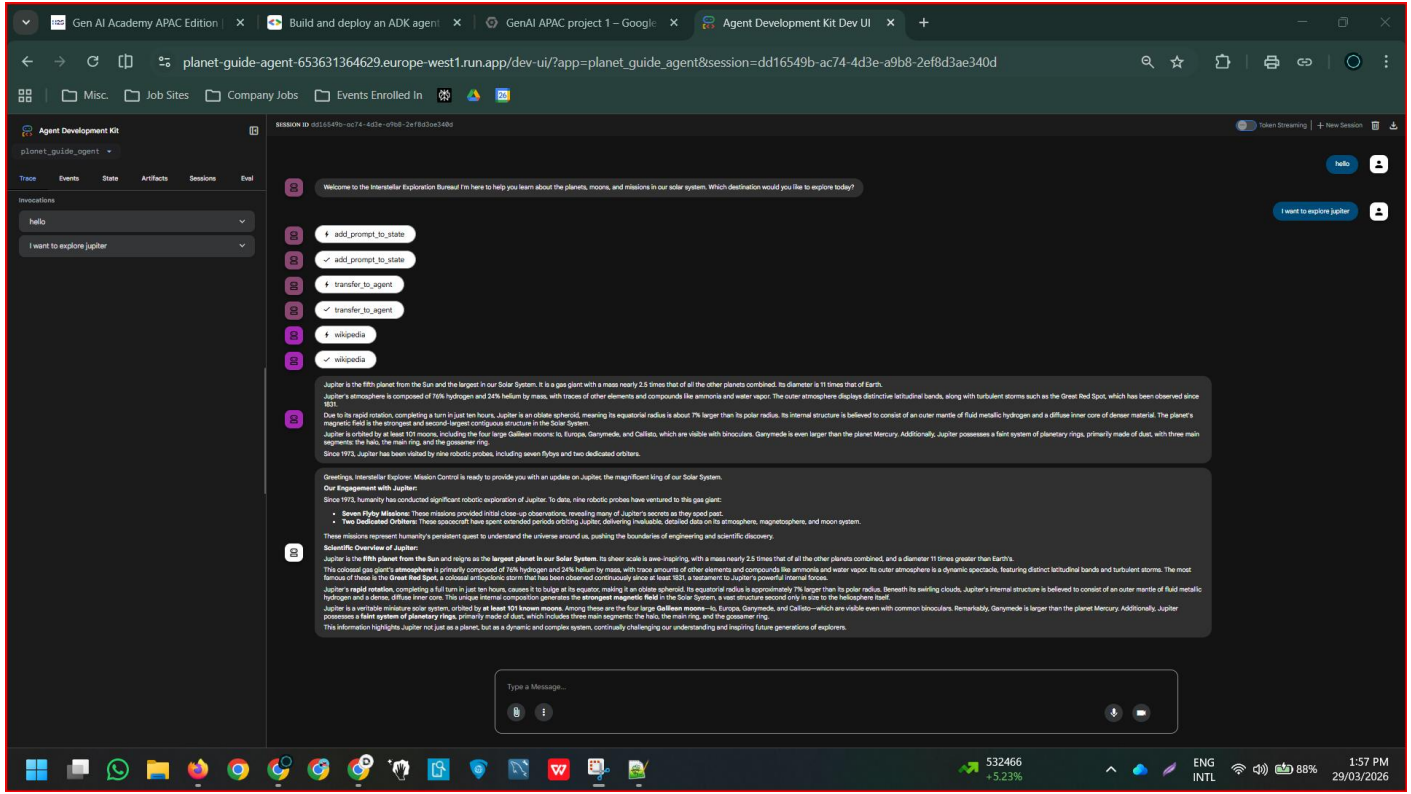
Process flow diagram or Use-case diagram

- Step 1 – User Interaction
  User opens the web UI and sends a question about a planet or moon (e.g. "Tell me about Mars").
- Step 2 – Greeter Agent (mission_control_greeter)
  Welcomes the user, captures their query, and saves it to shared agent state using the add_prompt_to_state tool.
- Step 3 – Handoff to Workflow
  Greeter transfers control to the planet_explorer_workflow (SequentialAgent).
- Step 4 – Researcher Agent (planetary_researcher)
  Queries Wikipedia using the LangchainTool wrapper. Retrieves scientific facts (gravity, atmosphere, orbital period, etc.). Stores results in research_data state key.
- Step 5 – Formatter Agent (response_formatter)
  Reads research_data from state. Synthesises findings into a professional Mission Control briefing.
- Step 6 – Response Delivered
  Final answer presented to the user via the Cloud Run web UI.

Technologies to be used in the solution

Google Agent Development Kit (ADK): Core framework for building, orchestrating, and deploying multi-agent AI workflows.
Gemini (via Vertex AI / AI Platform): Large language model powering all three agents for reasoning, research, and response generation.
LangChain + Wikipedia API: Used via ADK's LangchainTool wrapper for real-time planetary knowledge retrieval.
Google Cloud Run: Serverless deployment platform hosting the agent with a public web UI.
Google Cloud Build & Artifact Registry: Used to build and store the container image during deployment.
Google Cloud Logging: Captures agent activity and tool usage logs in production.
Python 3.12 + dotenv: Application runtime and environment configuration.
Docker: Containerisation of the agent application for Cloud Run deployment.

Snapshots of the prototype



Google Cloud H2S

Gen AI Academy APAC Edition

Build in APAC. Build for the world.

Thank you